

Chapter 11: AI-Driven Molecular Docking

Chapter 10 closed by designing a real peptide sequence — ETFEEIWAKLPQS and four close variants — for a real, fixed backbone conditioned on a real target interface (the MDM2-p53 cleft, PDB 1YCR), and asked the question that naturally follows: does a designed or selected binder actually *dock* into the site it was built or chosen for, in a way a computational method can predict before anything is synthesized? Part III (Chapters 8-10) was about producing 3D structure — of a single protein, or of a newly designed one. Part IV, which this chapter opens, is about the geometric relationship *between two* structures: a small molecule and the protein it may or may not bind, and where and how well it fits. That problem is **molecular docking**, and it has two methodologically distinct families of solution — a decades-old physics-based search paradigm (Section 11.1) and a recent generative deep-learning paradigm (Section 11.2) — that this chapter compares directly, both in what the literature reports (Section 11.3) and in what this chapter’s own hands-on project measures firsthand (Section 11.4) by docking dozens of real, potency-labeled compounds against a real oncology target.

11.1 Physics-Based Docking

Given a receptor structure and a candidate ligand, docking answers two coupled questions at once: what 3D pose (position, orientation, and internal conformation) should the ligand adopt relative to the receptor, and how favorable is that pose energetically? **AutoDock Vina** (Trott & Olson, 2010; Eberhardt et al., 2021) is the field’s most widely used open-source answer to both, and this chapter’s own hands-on project runs it directly, not just describes it.

The scoring function. Vina scores a pose with an empirical function built from pairwise atom-type interaction terms, summed over every pair of atoms i (receptor) and j (ligand) closer than a distance cutoff, as a function of the *surface* distance between them, $d_{ij} = r_{ij} - R_i - R_j$ (center-to-center distance minus the two atoms’ van der Waals radii — a physically motivated choice, since it is surface proximity, not nuclear distance, that governs steric and electrostatic contact):

$$c = \sum_{i < j} w_{t_i t_j} \cdot \left[f_{\text{gauss1}}(d_{ij}) + f_{\text{gauss2}}(d_{ij}) + f_{\text{repulsion}}(d_{ij}) + f_{\text{hydrophobic}}(d_{ij}) + f_{\text{Hbond}}(d_{ij}) \right]$$

where each f is a simple, smooth function of d_{ij} alone (two Gaussians of different widths favoring close steric contact at two characteristic distances, a quadratic penalty for any overlap $d_{ij} < 0$, and two linear step functions rewarding hydrophobic contact and directional hydrogen bonding within their own characteristic distance ranges) and $w_{t_i t_j}$ is a per-atom-type-pair weight fit once, globally, against a large set of experimentally measured protein-ligand complexes — not fit per target. The intermolecular score c is then converted to a predicted binding affinity via

$$\Delta G_{\text{pred}} = \frac{c}{1 + w_{\text{rot}} N_{\text{rot}}}$$

where N_{rot} is the number of the ligand’s active rotatable bonds and w_{rot} (also globally fit) penalizes flexible ligands for the conformational entropy they lose on binding — a coarse but real physical correction: two ligands with an identical raw contact score are not equally favorable to bind if one must freeze far more internal rotational freedom to do so.

Conformational search. Scoring one pose is cheap; finding the best one is the actual computational problem. A drug-like ligand’s pose space includes 3 translational, 3 rotational, and N_{rot} torsional degrees of freedom — a continuous space with no closed-form optimum and many local minima. Vina searches it with an iterated local-search strategy: repeatedly perturb the current pose at random (a Monte-Carlo-style step), locally optimize the perturbed pose with the Broyden-Fletcher-Goldfarb-Shanno (BFGS) quasi-Newton method (using the scoring function’s analytically computed gradient, which its smooth, piecewise-simple terms make cheap), and accept or reject the new local optimum with a Metropolis-style criterion, repeating for a number of independent runs controlled by the **exhaustiveness** parameter — directly trading wall-clock time for a higher chance of finding the true global optimum rather than a nearby local one. This chapter’s hands-on project uses Vina’s own documented default, `exhaustiveness = 8`, and reports the real wall-clock cost that choice carries (Section 11.4).

Pocket-informed vs. blind search. Vina always searches inside an axis-aligned rectangular box the user supplies — it has no built-in notion of “the binding site” beyond that box. When a binding pocket is already known (from a co-crystallized ligand, a homologous structure, or a cavity-detection tool), the box can be centered tightly on it — a few thousand cubic angstroms, a **focused** search. When it is not, the only physically honest option is a box large enough to cover the receptor’s entire solvent-accessible surface — a **blind** search, often tens of thousands of cubic angstroms, which Vina’s own runtime output explicitly flags as exceeding its recommended volume (Section 11.4 reports this warning firsthand). The same scoring function and search algorithm run in both cases; only the volume the search has to cover changes, and Section 11.3 quantifies what that costs.

11.2 Deep Learning Docking

Physics-based search treats every new ligand-receptor pair as a fresh optimization problem, with no memory of any pose it has scored before. A second family of methods instead trains a neural network, once, on a large set of experimentally solved protein-ligand complexes, and asks it to output a pose directly — trading Vina’s per-complex search cost for a single learned forward (or iterative denoising) pass.

EquiBind (Stärk et al., 2022) frames docking as one-shot geometric *regression*: an $SE(3)$ -equivariant graph neural network — a network whose predictions rotate and translate exactly as its input coordinates do, the same equivariance principle Chapter 6 §6.4 introduced for property prediction, here applied to pose generation directly — jointly predicts a set of ligand and receptor “keypoints” and the optimal rigid transformation (rotation and translation) aligning one point set to the other, then relaxes the ligand’s internal torsion angles to resolve steric clashes. Being a single deterministic forward pass, EquiBind is extremely fast, but a regression model must commit to one geometric answer per input — it has no native mechanism for expressing “there are two comparably plausible binding modes here,” a limitation the next two methods address by construction, from opposite directions.

TankBind (Lu et al., 2022) keeps prediction one-shot but reframes its *target*: rather than regress atomic coordinates directly, it segments the receptor surface into candidate functional blocks

(pocket-sized regions), predicts, for each block, a full pairwise distance matrix between every ligand atom and every block residue, and reconstructs the 3D pose from that distance matrix by distance geometry — a trigonometry-aware architecture that explicitly enforces the triangle inequality between predicted distances during training (hence the name), which plain per-atom coordinate regression has no mechanism to guarantee. Scoring every candidate block against the same ligand also gives TankBind a natural, built-in way to ask *which* site a ligand most likely binds, not only *how* it binds a site assumed known in advance.

DiffDock (Corso et al., 2023) instead frames docking as *generative* modeling over the manifold of possible poses, applying the same denoising-diffusion principle Chapter 7 §7.4 used for generating 3D molecular structures from nothing and Chapter 10 §10.1 used for generating protein backbones — here, to generating a ligand pose conditioned on a fixed receptor. A pose’s true degrees of freedom are not $3N$ independent atomic coordinates but exactly the low-dimensional product space this chapter’s own Section 11.1 named for physics-based search: translation in $\mathbb{R}^3$, global rotation in $SO(3)$, and N_{rot} torsion angles each in $SO(2)$. DiffDock defines its forward noising process, and therefore the reverse denoising process a trained score network learns to run, directly on this product manifold rather than on raw coordinates — a substantially lower-dimensional space to search than physics-based methods implicitly explore atom by atom, and a *generative* one: sampling the reverse process repeatedly from different random starting poses yields multiple, diverse candidate poses per complex rather than regression’s single deterministic answer, together with a learned confidence model that ranks them. Corso et al. (2024) subsequently released DiffDock-L, an updated model trained on substantially more data with generalization-focused refinements to the same underlying architecture; this chapter’s citations and comparisons refer to the original DiffDock unless noted otherwise, since it is the version with the longest-standing, most widely reproduced benchmark record.

A feasibility investigation, before any code was written. Before deciding whether to run DiffDock in this chapter’s hands-on project (Section 11.4), its real installation and inference requirements were checked directly, the same way Chapter 10 §10.1 checked RFDiffusion’s. The official gcorso/DiffDock repository documents environment setup through a `conda env create` command pinned to a `torch/torch-geometric/e3nn` stack, recommends GPU inference explicitly (“we recommend using a GPU as the model runs significantly faster”), and offers no batch programmatic API beyond its own command-line inference script and dataset-CSV interface — the only zero-install option is a public Hugging Face Space (`reginabarzilaygroup/DiffDock-Web`) whose own documentation states it “is designed to take 1 protein... and 1 ligand... at a time” and explicitly directs bulk use back to the command-line interface; querying that Space directly while preparing this chapter returned an HTTP 503 (service unavailable), consistent with it being a best-effort community demo rather than a stable, reproducible batch endpoint. This is the same category of constraint Chapter 10 hit with RFDiffusion: no `pip install`-only path, a GPU recommendation this book’s CPU-only authoring environment cannot satisfy, and no lightweight hosted API standing in for it the way the ESM Metagenomic Atlas API did for ESMFold in Chapters 9-10. Section 11.4’s hands-on project therefore runs real physics-based (Vina) docking end to end and discusses DiffDock’s reported benchmark numbers as literature (Section 11.3), rather than re-running DiffDock itself — stated here explicitly rather than silently substituted.

11.3 Speed vs. Accuracy Trade-offs

The two paradigms just described are not merely different implementations of the same task; the published record shows them trading speed and accuracy against each other in opposite directions, and this chapter separates what the literature reports about that trade-off from what its own hands-on project independently measures.

What the literature reports. Benchmarking on PDBBind (the standard curated set of experimentally solved protein-ligand complexes docking methods are evaluated against), Corso et al. (2023) report DiffDock reaching a **38% top-1 success rate** (fraction of complexes whose top-ranked predicted pose falls within 2 Å heavy-atom RMSD of the true crystal pose) — against **23%** for the traditional physics-based docking methods and **20%** for prior regression-based deep learning methods (EquiBind’s own paradigm) they compare against in the same evaluation. The gap widens further on *computationally predicted* receptor structures rather than experimental ones (i.e., an AlphaFold2- or ESMFold-predicted structure standing in for a crystal structure, the realistic scenario for any target without its own solved structure): prior methods reach at most 10.4% success, while DiffDock reaches 21.7% — evidence that a learned pose distribution generalizes to structural noise more gracefully than a rigid physics-based search does. Separately, DiffDock’s diffusion sampling is reported to run in roughly seconds to tens of seconds per complex on GPU — the source of deep-learning docking’s headline speed advantage over exhaustive physics-based search, though a fair comparison requires a like-for-like hardware basis (GPU inference vs. CPU search) that published comparisons do not always make explicit.

What this chapter’s own experiment measures. Section 11.4 runs the one paradigm that is feasible to execute directly in this book’s authoring environment — real AutoDock Vina, on real CPU hardware, against a real oncology target — under both the focused and blind search-box conditions Section 11.1 distinguished, and reports real wall-clock time and pose agreement between them. This is not a replication of DiffDock’s own benchmark (a GPU-based, learned-model comparison this environment cannot run, per Section 11.2’s feasibility finding) but a direct, real measurement of physics-based search’s own internal speed/accuracy trade-off — exactly what changes, and by how much, when the same scoring function and search algorithm are asked to cover a receptor’s full surface instead of a known pocket.

11.4 Hands-on Project: Real AutoDock Vina Docking Against a Real Oncology Target

The project code lives in this chapter’s folder (`ch11_molecular_docking/`). Given Section 11.2’s feasibility finding, this project runs real, physics-based AutoDock Vina docking end to end — real receptor and ligand preparation, real scoring, and a real redocking validation control — rather than re-running DiffDock.

Real data: one real receptor, a real curated ChEMBL benchmark set

PDB 1M17 (Stamos, Sliwkowski & Eigenbrot, 2002): the EGFR tyrosine kinase domain in complex with **erlotinib**, a real, clinically approved non-small-cell-lung-cancer drug (PDB heteroatom code AQ4) — continuing this book’s EGFR thread from Chapters 1 and 3, and a well-characterized

oncology target with an unambiguous, experimentally defined binding pocket this project uses for its redocking control and its focused-search box.

A real, curated ChEMBL EGFR benchmark set. `molecular_docking.py` fetches real, measured IC50 binding-assay records for EGFR (ChEMBL target CHEMBL203, human) live from the ChEMBL REST API (Mendez et al., 2019) — 18,123 records carried a measured pChEMBL value at the time this chapter was written, of which the first 1,000 were fetched (a bounded page, sufficient for a diverse curated subset; see `fetch_egfr_activities`). These are deduplicated to one entry per real molecule (median pChEMBL across repeated measurements), filtered to single-fragment, RDKit-sanitizable structures under 700 Da (the same drug-likeness sanity filter Chapter 2 §2.5 applied via Lipinski’s Ro5), and deterministically stratified-sampled to **30 compounds evenly spaced across the real measured potency range** — from the pool’s weakest to its most potent measured EGFR binder, not an arbitrary or convenience subset. The curated set and its provenance are cached in `data/egfr_chembl_benchmark.json` for offline reproducibility.

Why 30 compounds, not the outline’s illustrative 1,000. Docking this real 30-compound set under both conditions (60 real Vina runs) was timed directly: a mean of 568 s (focused) and 576 s (blind) of wall-clock CPU time per run — roughly 9.5 minutes each, and substantially higher than either condition’s per-ligand cost measured in isolation earlier in this project’s development (roughly 1-3 minutes per run with no other Vina process competing for the same cores). The real, measured cause is resource contention: this chapter’s own code runs four docking jobs in parallel (`--n-workers 4`, one per CPU core) to keep total wall-clock time down, and four independent Vina searches sharing the same small number of physical cores each individually slow down under that real contention — a genuine, disclosed systems-engineering cost of parallelizing on limited hardware, not a property of Vina’s algorithm itself. At that real, measured per-run cost, docking 1,000 compounds under both conditions this chapter compares (2,000 real Vina runs) would need on the order of 300 CPU-hours — far beyond what this book’s free-tier-Colab-compatible authoring environment can reasonably absorb in one working session; the outline’s own Chapter 13 (High-Throughput Virtual Screening) exists specifically to address campaigns at that scale, via the tiered fast-filter-then-dock-then-simulate funnel this chapter’s smaller, fully-executed benchmark foreshadows rather than replaces. Thirty real, potency-labeled compounds are enough to compute a real Spearman correlation and a real focused-vs-blind agreement statistic with reasonable statistical power, while remaining honestly executable within this environment — the same “shrink the real experiment rather than fabricate the large one” choice Chapter 10 made for its own compute-bound hands-on project.

Real receptor and ligand preparation

The real 1M17 structure is split into a protein-only file (all ATOM records) and the native ligand’s HETATM records, and the receptor is protonated at pH 7.4 and converted to a rigid AutoDock PDBQT file with **OpenBabel** (O’Boyle et al., 2011) — the standard simplified rigid-receptor protocol most Vina pipelines use, distinct from (and far cheaper than) predicting explicit side-chain flexibility. Each ligand is embedded as a real 3D conformer from its ChEMBL SMILES string with RDKit’s ETKDG algorithm (Chapter 2 §2.4) and MMFF94-optimized, then converted to a PDBQT file — rotatable-bond torsion tree and Gasteiger partial charges assigned automatically

— with **Meeko** (github.com/forlilab/meeko), the docking-preparation toolkit maintained by the same Scripps Research lab that develops AutoDock Vina itself:

```
def prepare_ligand_pdbqt(smiles: str, seed: int = 42) -> str | None:
    """Embed a real 3D conformer (ETKDG + MMFF94) and convert it to an
    AutoDock PDBQT string via Meeko. Returns None if 3D embedding fails."""
    mol = Chem.AddHs(Chem.MolFromSmiles(smiles))
    if AllChem.EmbedMolecule(mol, randomSeed=seed, useRandomCoords=True) < 0:
        return None
    AllChem.MMFFOptimizeMolecule(mol)
    mol_setups = MoleculePreparation().prepare(mol)
    pdbqt_string, is_ok, _ = PDBQTWriterLegacy.write_string(mol_setups[0])
    return pdbqt_string if is_ok else None
```

Docking itself calls the real, official AutoDock Vina engine — the vina Python bindings when importable (the zero-friction path a Colab or Linux reader gets from `pip install vina`), falling back to the official standalone Vina executable when the Python bindings are not installed for the host platform. This is a real, disclosed environment constraint rather than a hypothetical one: the vina PyPI package ships prebuilt wheels only for manylinux/musllinux/macOS, with no Windows wheel, and building it from source requires the Boost C++ libraries; this chapter’s own results were generated by exercising the CLI-binary fallback path directly, invoking the identical official `ccsb-scripps/AutoDock-Vina v1.2.7` release binary that `pip install vina` itself wraps in Boost.Python on supported platforms — the same underlying scoring and search engine either way, confirmed directly against the output format both paths produce (every pose’s affinity appears as an identical `REMARK VINA RESULT:` line regardless of which entry point wrote it).

Real docking boxes, computed from real geometry

The focused search box is a 22.5-Å cube centered on erlotinib’s real crystallographic centroid (22.01, 0.25, 52.79); the blind search box covers the receptor’s own real atomic bounding box plus a 6-Å margin on every side — $104.2 \times 77.9 \times 63.4$ Å, roughly 517,000 Å³, over nineteen times the focused box’s volume and a volume Vina’s own runtime output flags directly (“WARNING: Search space volume is greater than 27000 Angstrom^3 (See FAQ)”), a real, first-hand confirmation of Section 11.1’s point that blind search trades defined volume for an explicit, tool-reported reliability caveat.

Redocking validation

Before docking any ChEMBL compound, erlotinib is docked back into its own real crystallographic pocket and the resulting top pose’s heavy-atom RMSD to the real 1M17 crystal structure is computed (RDKit’s `GetBestRMS`, which handles atom-index correspondence and molecular symmetry automatically) — the standard redocking self-consistency control the field uses to validate a docking setup before trusting it on any compound without a known answer, against the literature’s conventional <2.0-Å “correct pose” threshold (unrelated to any arbitrary choice made elsewhere in this chapter). This chapter runs the control **five independent times** (different random seeds each time), because an early, real observation made while developing this project was that AutoDock Vina’s own reported affinity and pose RMSD are not perfectly reproducible run to run even with a fixed `--seed` and single-threaded (`--cpu 1`) execution, with receptor and

ligand PDBQT preparation both independently confirmed byte-identical across repeated runs — reporting a distribution is the honest response to that real finding, not a single, possibly favorable, point estimate:

Replicate	Affinity (kcal/mol)	RMSD to crystal pose (Å)	Correct pose (<2.0 Å)?
1	-7.678	2.604	No
2	-7.735	2.632	No
3	-7.475	2.166	No
4	-7.700	2.617	No
5	-7.619	2.611	No
Mean ± SD	-7.641 ± 0.091	2.526 ± 0.180	0/5

Erlotinib’s real, correctly-favorable (negative) predicted binding affinity is reproduced consistently and with low variance across replicates (SD 0.09 kcal/mol) — the search reliably finds a strongly-scoring pose in the right general region of the pocket. Its RMSD to the crystal pose, however, consistently lands just *above* the field’s conventional 2.0-Å “correct pose” threshold (2.17-2.63 Å across all five replicates, none passing) rather than comfortably below it. This is reported as the real result, not adjusted or reframed as a near-miss: with a fully rigid receptor (no side-chain flexibility), a simplified Gasteiger-charge electrostatic model, and Vina’s own default exhaustiveness, this docking setup consistently finds a pose in erlotinib’s real binding pocket but not, by the field’s own strict convention, the *exact* crystallographic orientation — a genuine, disclosed limitation of the simplified protocol used throughout this chapter’s hands-on project, not a bug, and the reason the ChEMBL benchmark results below are interpreted as evidence about relative ranking and pocket identification rather than sub-2-Å pose accuracy.

Docking the real benchmark set: results

Docking all 30 real curated compounds against the real EGFR receptor, under both the focused and blind conditions, succeeded for every compound (30/30; one transient single-run Vina failure did occur mid-campaign under the four-way parallel CPU contention just described, handled by this chapter’s own retry/resume logic — see `molecular_docking.py`’s `run_vina_docking`, and the “A real mid-run failure” note below):

Statistic	Focused	Blind
Mean affinity (kcal/mol)	-8.655 ± 0.758 (range -11.065 to -7.618)	—
Mean wall time (s)	568.4	576.3
Total wall time (s)	17,053	17,287

Docking score vs. experimental potency: Spearman $\rho = 0.245$ ($p = 0.19$, $n = 30$) — not statistically significant. This is the honest, reportable result, not a disappointing one to explain away: AutoDock Vina’s empirical scoring function is well documented in the primary literature (Section 11.1) as a *pose-finding* energy function fit globally across many complexes, not a compound-ranking affinity predictor for one target’s structure-activity series, and this chapter’s own real data illustrates exactly that distinction. The single most striking example: **CHEMBL62843**, the *weakest* measured binder in the entire curated set (pChEMBL 4.00, roughly 100 μM), receives the *most* favorable focused-docking affinity of all 30 compounds (-11.065 kcal/mol) — it is also, at 594.7 Da, the heaviest and most polar-surface-rich compound in the set, and Vina’s scoring terms (Section 11.1) reward extensive favorable atom-pair contacts somewhat independently of whether those contacts correspond to the specific chemistry that makes a real inhibitor potent. This is a real, literature-consistent limitation of empirical scoring functions (the same one motivating deep-learning approaches trained end-to-end for pose *and* affinity, Section 11.2) rather than a defect in this chapter’s implementation.

Focused vs. blind agreement: Spearman $\rho = 0.882$ ($p < 0.001$) between the two conditions’ affinities, and a median top-pose centroid distance of only 0.49 Å (mean 1.58 Å; 93.3% of compounds land within 5 Å of their own focused-search pose). For this particular target — a deep, well-defined ATP-competitive kinase pocket, not a shallow or multi-site surface — blind search across the receptor’s entire real bounding box (Section 11.1’s ~517,000-Å box, over nineteen times the focused box’s volume) converges to essentially the same site and pose the pocket-informed search finds, for the large majority of real compounds tested, despite Vina’s own runtime warning about that box’s size. This is a genuinely positive, real finding about blind docking’s practical reliability *for this specific, well-characterized pocket type* — it should not be read as a general claim that blind docking reliably finds the correct site on any target, since a shallow or multi-pocket surface would not necessarily replicate it.

A real mid-run failure, handled rather than hidden. During this benchmark’s execution, one of the 60 real Vina subprocess calls exited non-zero under the four-way parallel contention described above (real stderr captured, not fabricated); re-running that exact real receptor/ligand/seed combination in isolation immediately afterward succeeded cleanly (-11.07 kcal/mol, consistent with the -11.065/-11.128 kcal/mol this same compound’s focused/blind runs eventually recorded), confirming the failure was a transient resource-contention effect, not a property of that molecule or a bug in this chapter’s chemistry pipeline. `molecular_docking.py` records such failures per-compound (rather than letting one bad subprocess call crash the entire `ProcessPoolExecutor` pool and discard every other compound’s already-completed real result) and supports resuming a run from whatever real, valid PDBQT output already exists on disk — an engineering property any long-running real docking campaign needs, not a workaround specific to this one incident.

Reproducibility

Dependencies are version-floored (`rdkit` ≥ 2023.9 , `meeko` ≥ 0.6 , `numpy` ≥ 1.24 , `scipy` ≥ 1.10 , `requests` ≥ 2.28 in `requirements.txt`; `vina` $\geq 1.2.5$ on platforms with a prebuilt wheel — validated against `rdkit` 2026.03.5, `meeko` 0.7.1, `numpy` 2.5.2, `scipy` 1.18.0, `requests` 2.34.2, and `OpenBabel` 3.1.0 on Python 3.12). `data/egfr_chembl_benchmark.json` caches the exact real curated

compound set this chapter’s results were computed from, so `python molecular_docking.py` reproduces the same 30 compounds offline without a live ChEMBL API call (pass `--refresh-cache` to re-curate from a fresh live fetch instead). The `tests/test_molecular_docking.py` suite exercises real receptor/ligand preparation and the real ChEMBL curation logic directly against bundled fixtures; the small number of tests that would otherwise invoke Vina itself are skipped automatically when no Vina engine (Python bindings or executable) is available on the host, following the same “real where feasible, honestly skipped where not” principle as the rest of this book’s test suites.

Limitations and what comes next

This project validates real, physics-based AutoDock Vina docking against one real, well-characterized oncology target with a real redocking control and a real, potency-labeled compound benchmark — it does not run or re-benchmark any deep-learning docking method itself, for the feasibility reasons Section 11.2 documents explicitly, and its 30-compound benchmark is a real, honestly-scoped subset rather than the outline’s illustrative 1,000-compound figure, for the compute-time reasons documented above. Rigid-receptor docking also cannot model genuine induced-fit conformational change on binding — a real physical limitation of the simplified protocol used here, not specific to this chapter’s implementation. Chapter 12 picks up the natural next question this limitation raises directly: given a predicted pose, is it actually *stable* once real molecular motion and solvent are simulated, rather than frozen at whatever single conformation docking returned?

A note on Google Colab

`rdkit`, `numpy`, `scipy`, and `requests` are preinstalled on Colab’s default runtime; `meeko` and `vina` need `!pip install meeko vina`, and `OpenBabel` needs `!apt-get install -y openbabel` (a system package, not a Python one, since this chapter uses `OpenBabel`’s command-line tool directly). No GPU is required — every step in this hands-on project runs on CPU, exactly as it was run to produce the results above.

References

- Trott, O., & Olson, A. J. (2010). AutoDock Vina: Improving the speed and accuracy of docking with a new scoring function, efficient optimization, and multithreading. *Journal of Computational Chemistry*, 31(2), 455-461. <https://doi.org/10.1002/jcc.21334>
- Eberhardt, J., Santos-Martins, D., Tillack, A. F., & Forli, S. (2021). AutoDock Vina 1.2.0: New docking methods, expanded force field, and Python bindings. *Journal of Chemical Information and Modeling*, 61(8), 3891-3898. <https://doi.org/10.1021/acs.jcim.1c00203>
- Stärk, H., Ganea, O., Pattanaik, L., Barzilay, R., & Jaakkola, T. (2022). EquiBind: Geometric Deep Learning for Drug Binding Structure Prediction. *Proceedings of the 39th International Conference on Machine Learning (ICML)*, PMLR 162, 20503-20521.
- Lu, W., Wu, Q., Zhang, J., Rao, J., Li, C., & Zheng, S. (2022). TANKBind: Trigonometry-Aware Neural Networks for Drug-Protein Binding Structure Prediction. *Advances in Neural Information Processing Systems (NeurIPS)* 35. Preprint: bioRxiv 2022.06.06.495043. <https://doi.org/10.1101/2022.06.06.495043>

- Corso, G., Stärk, H., Jing, B., Barzilay, R., & Jaakkola, T. (2023). DiffDock: Diffusion Steps, Twists, and Turns for Molecular Docking. *International Conference on Learning Representations (ICLR)*. arXiv:2210.01776. <https://arxiv.org/abs/2210.01776>
- Corso, G., Deng, A., Polizzi, N., Barzilay, R., & Jaakkola, T. (2024). Deep Confident Steps to New Pockets: Strategies for Docking Generalization. *International Conference on Learning Representations (ICLR)*. arXiv:2402.18396. <https://arxiv.org/abs/2402.18396>
- Stamos, J., Sliwkowski, M. X., & Eigenbrot, C. (2002). Structure of the epidermal growth factor receptor kinase domain alone and in complex with a 4-anilinoquinazoline inhibitor. *Journal of Biological Chemistry*, 277(48), 46265-46272. <https://doi.org/10.1074/jbc.M207135200>
- O’Boyle, N. M., Banck, M., James, C. A., Morley, C., Vandermeersch, T., & Hutchison, G. R. (2011). Open Babel: An open chemical toolbox. *Journal of Cheminformatics*, 3, 33. <https://doi.org/10.1186/1758-2946-3-33>

See Chapter 1’s references for Mendez et al. (2019, ChEMBL), reused directly for this chapter’s live bioactivity data source. RDKit (Chapters 2, 4) and Meeko (github.com/forlilab/meeko, no separate primary paper) are used for ligand preparation but play no scoring role of their own — AutoDock Vina’s scoring function and search algorithm, described in Section 11.1, are unmodified.

All affinities, timings, RMSD values, and correlation statistics cited in Section 11.4 were computed directly by running `molecular_docking.py` against the real bundled PDB 1M17 structure and the real curated ChEMBL benchmark set on 2026-08-20/21 (the real 60-run docking campaign’s wall-clock cost, detailed above, spanned the two calendar dates), not taken from a secondary source — see `results/molecular_docking_results.json` to reproduce.